COS 433: Cryptography

Announcements:

- Problem Set 4 due Wed 11/11

Digital signatures

One-Time Signatures



$\mathrm{Gen}(1^\lambda) \to (\mathrm{sk}, \mathrm{vk})$



$m \in \{0,1\}^n$

vk (broadcast)

$(m, \sigma = \mathrm{Sign}(\mathrm{sk}, m))$



$\mathrm{Verify}(\mathrm{vk}, m, \sigma) \in \{0,1\}$

Theorem: Assuming one-way functions exist, for any $n(\lambda) = \mathrm{poly}(\lambda)$, there exist one-time signatures with message length $n(\lambda)$!

- Key length: $|\mathrm{vk}| = n(\lambda) \cdot \lambda > n(\lambda)$ (long keys)
- We will fix this ~~later~~ now

One-time signatures with short keys

Hash-and-Sign Paradigm



$\text{Gen}(1^\lambda) \rightarrow (\text{sk}, \text{vk})$



$m \in \{0,1\}^n$

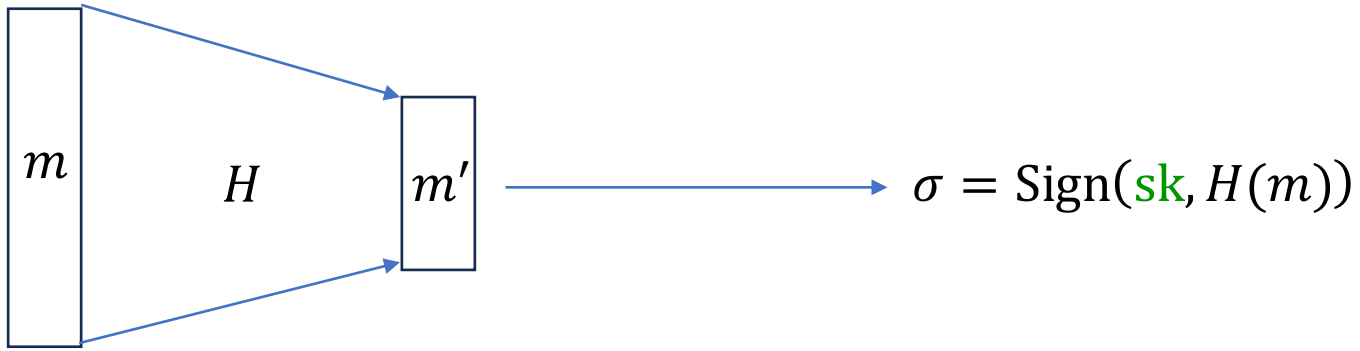
vk (broadcast)

$(m, \sigma = \text{Sign}(\text{sk}, m))$



$\text{Verify}(\text{vk}, m, \sigma) \in \{0, 1\}$

- Idea: extend the scheme to handle longer messages with (roughly) same vk



- H must be a publicly computable "hash function" (so that σ can be verified)

Hash-and-Sign Paradigm



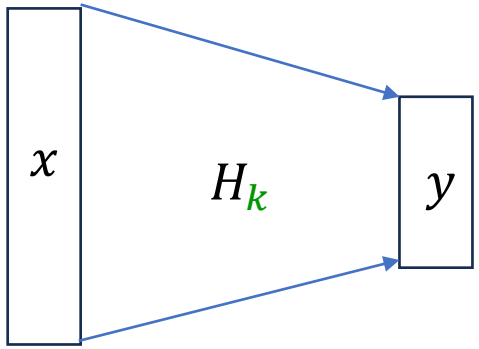
- Is this secure?
- If H is a fixed function: no!
  - Suppose that $H(m_1) = H(m_2)$. Then can forge on m_2 given signature on m_1.

Hash-and-Sign Paradigm



- Is this secure?
- If H is a fixed function: no!
  - Suppose that $H(m_1) = H(m_2)$. Then can forge on m_2 given signature on m_1.
- Instead: use a keyed hash function H_k
  - Add k to both the signing and verification keys

Collision-Resistant Hash Functions



Definition: A family of efficiently computable functions $\mathcal{H} = \{H_k : \{0,1\}^{n(\lambda)} \to \{0,1\}^{\lambda}\}_{k \in K}$ is a collision-resistant hash function (CRHF) family with input length $n = n(\lambda)$ if for all polynomial-time A,

$$\Pr[A(k) \to (x_1, x_2) : x_1 \neq x_2 \text{ and } H_k(x_1) = H_k(x_2)] = \text{negl}(\lambda)$$

Collision-Resistant Hash Functions



Theorem: If

- $(\mathrm{Gen}, \mathrm{Sign}, \mathrm{Verify})$ is a (one time OR full) digital signature scheme with message length $n = n(\lambda)$, and
- $\mathcal{H}$ is a collision-resistant hash function (CRHF) family with input length n,

then $(\mathrm{Sign} \circ \mathcal{H}, \mathrm{Verify} \circ \mathcal{H})$ is also a (one time OR full) digital signature scheme with

$$\mathrm{Gen}'(1^\lambda) \to ((\mathrm{sk}, k), (\mathrm{vk}, k))$$

Collision-Resistant Hash Functions



So if we can write down a CHRF family with short hash keys, we can sign (single) long messages with short (sk, vk)!

Intuition: $H_k = \text{PRF}_k$ would work… if k were secret.

Two kinds of constructions

- Algebraic CRHFs based on, e.g., discrete log
- "Obscure programs" approach with a heuristic analysis: random oracle model

CRHFs based on discrete log

$H_k : \mathbb{Z}_q \times \mathbb{Z}_q \to \mathbb{Z}_p$

$p = 2q + 1$

$k = (g, h)$

g generator for $\mathbf{QR}_p \subset \mathbb{Z}_p^\times$

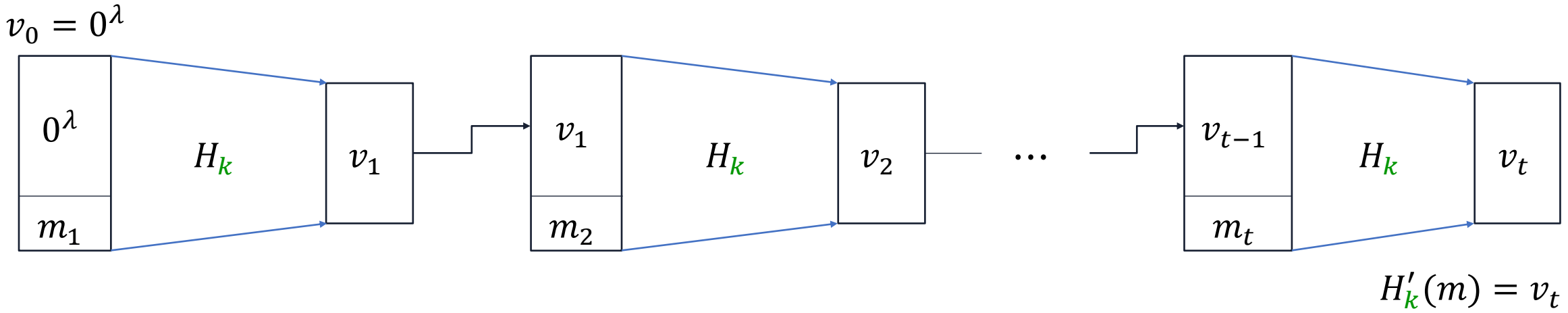
$h \leftarrow \mathbf{QR}_p$

$$H_k(x, y) = g^x h^y$$

CRHF Length Extension: Merkle–Damgård

$$H_k : \{0,1\}^{\lambda+1} \rightarrow \{0,1\}^{\lambda}$$

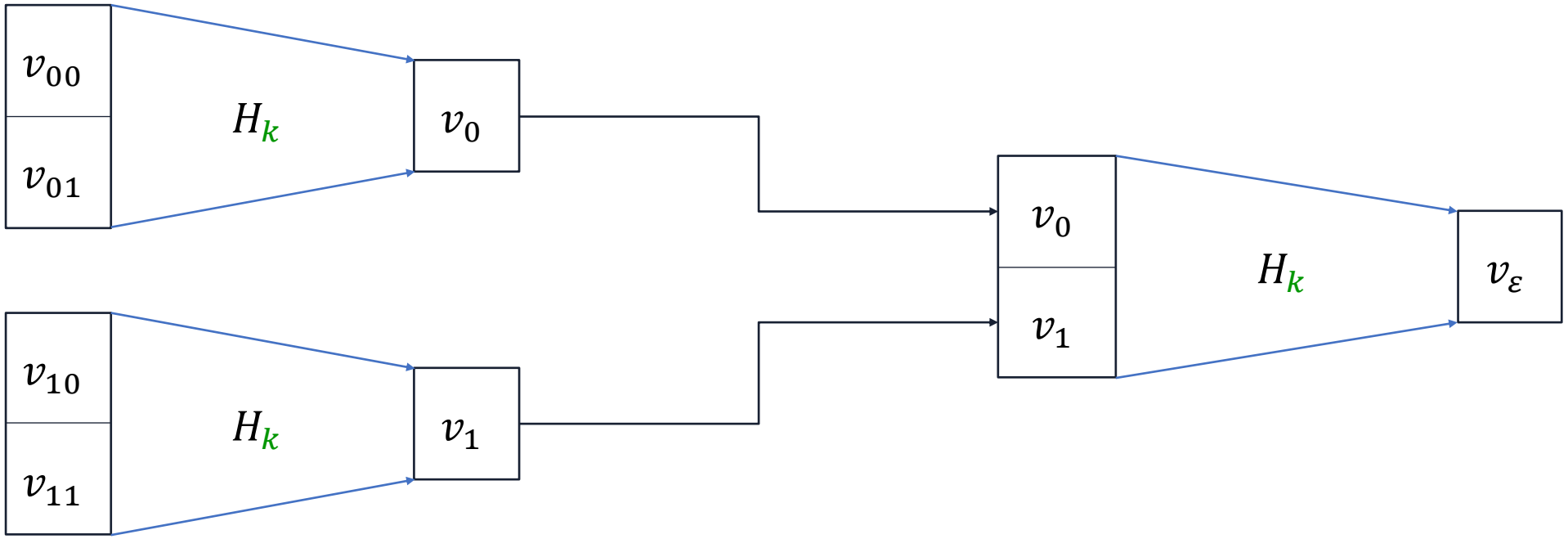
$$m = m_1 \parallel m_2 \parallel \cdots \parallel m_t, m_i \in \{0,1\}$$



$$v_i = H_k(v_{i-1} \parallel m_i)$$

CRHF Length Extension: Merkle Trees

$$H_k : \{0,1\}^{2\lambda} \rightarrow \{0,1\}^{\lambda}$$

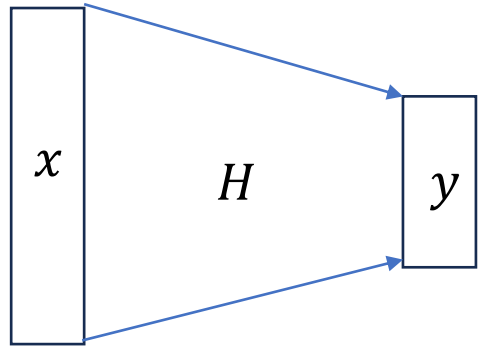


$$m = v_{00} \parallel v_{01} \parallel v_{10} \parallel v_{11}, v_s \in \{0,1\}^{\lambda}$$

Four block example

Random Oracle Heuristic

1. Design a crypto scheme that makes black-box use of a hash function H.



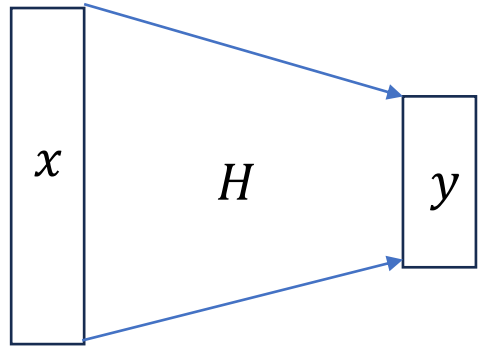
2. Prove security "in the random oracle model:"
   - H is a uniformly random function
   - Adversary is of the form $A^{H(\cdot)}$, making black-box calls to H.

3. Heuristically instantiate with a concrete hash function family $\{H_k\}$

4. Philosophy (unproven): carefully designed hash functions should not have non-black-box attacks.

Random Oracle Heuristic

1. Design a crypto scheme that makes black-box use of a hash function H.



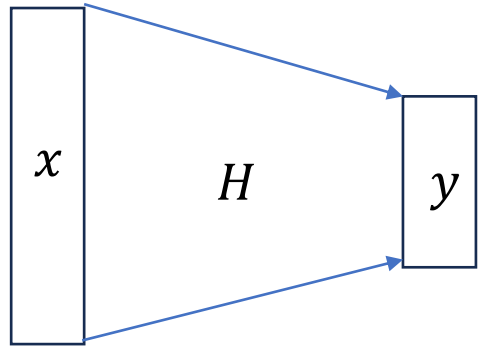
2. Prove security "in the random oracle model:"
   - H is a uniformly random function
   - Adversary is of the form $A^{H(\cdot)}$, making black-box calls to H.

3. Heuristically instantiate with a concrete hash function family $\{H_k\}$

4. Example: H is a CRHF in the random oracle model (either unkeyed or salted: $H_k(x) = H(k, x)$)

Random Oracle Heuristic

1. Design a crypto scheme that makes black-box use of a hash function H.



2. Prove security "in the random oracle model:"
   - H is a uniformly random function
   - Adversary is of the form $A^{H(\cdot)}$, making black-box calls to H.

3. Heuristically instantiate with a concrete hash function family $\{H_k\}$

Extremely common/powerful tool for designing practical cryptography

But: sometimes unsound, need to be careful

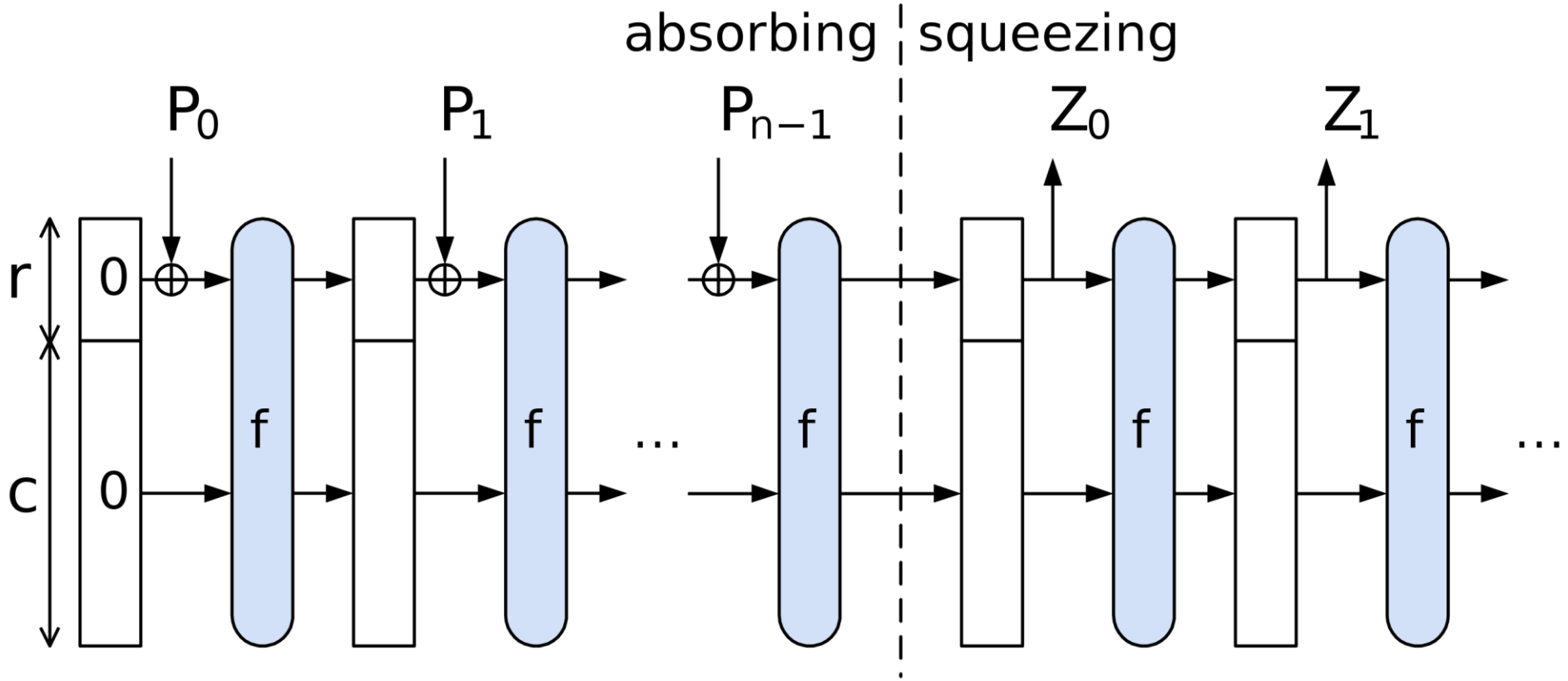
Random Oracle Instantiations (heuristic)

- SHA-2 (2001-)
  - Output length: 256 or 512 bits
  - Fixed-length hash function; use Merkle-Damgard to extend input length
    - This works for collision resistance but not every security property!
    - Need to worry about length extension attacks

- SHA-3 (2015-)
  - Output length: 256 or 512 bits
  - “Sponge construction”
  - Unbounded input length!

- Theoretical: “obfuscated pseudorandom function” (we won’t discuss)

SHA-3: the Sponge Construction



- f: public, AES-like permutation (composition of several simple permutations for 24 rounds)
  - Unlike AES: keyless!
- Proceeds block-by-block
  - Repeatedly: XOR in a block of your input, then apply f
  - Repeatedly: output a block of the current memory contents, then apply f

Applications of Random Oracles

- Two practical signature schemes: RSA and Schnorr/DSA

- Practical, CCA-secure public-key encryption (we won’t do this in class)

continued next time